ЛАБОРАТОРНАЯ РАБОТА №5
DAST: OWASP ZAP, проверка исправлений и CI/CD

Теория → пошаговая практика → проверка результата
2 академических часа · учебный проект vulnerable-app

Выполняем только на локальной учебной копии. Реальные секреты и чужие системы не используем.

1. Что вы изучите

Научиться проверять работающее приложение через ZAP, понимать ограничения DAST и объединять все проверки практикума в единый процесс.

• Запустите ZAP Baseline.

• Разберёте active scan и покрытие.

• Подтвердите SQLi, XSS и IDOR локально.

• Добавите контроль доступа и заголовки.

• Составите pipeline с quality gates.

Опыт программирования не требуется: команды и фрагменты кода можно копировать. После каждого шага объясняется, что должно получиться и как проверить результат.

2. Теория простыми словами

Что делает DAST

DAST отправляет запросы работающему приложению и анализирует ответы. Исходный код не нужен. DAST видит реальные заголовки и поведение, но только те маршруты и сценарии, которые смог пройти.

Baseline и active scan

Baseline использует spider и пассивные правила: он не проводит атаки. Active scan изменяет параметры и может создавать данные. Его разрешено запускать только на своей изолированной цели.

WARN, FAIL и покрытие

WARN — сигнал для разбора, а не готовое доказательство. FAIL: 0 не означает безопасность. Нужно проверить, какие страницы посещены, была ли аутентификация и какие роли проверены.

Почему IDOR ищут вручную

Сканер не знает, какой объект принадлежит пользователю. Если /profile?id=1 доступен без проверки владельца, это IDOR. Исправление выполняется backend: без сессии 401, чужой id 403.

Заголовки безопасности

CSP ограничивает источники содержимого, X-Content-Type-Options запрещает MIME-sniffing, X-Frame-Options защищает от clickjacking. Заголовки не заменяют исправление XSS и CSRF-токены.

CI/CD

На PR запускают быстрые Gitleaks, SAST и SCA. После развёртывания в staging выполняют DAST. SBOM сохраняют с артефактом. Ошибка сканера не считается успехом. ГОСТ-процесс требует не только инструменты, но и решения, владельцев и повторные проверки.

3. Перед началом

• Распакуйте практикум и откройте терминал в каталоге vulnerable-app.

• Нужны Python 3.11+, браузер и терминал. Для ZAP и альтернативных запусков — Docker Desktop.

• Команды показаны для Git Bash. Для PowerShell используйте .venv\Scripts\Activate.ps1 вместо source.

• Создайте папку reports для результатов. Не публикуйте отчёты, пока не проверили их содержимое.

• Сохраняйте исходную копию проекта, чтобы сравнивать состояние до и после исправлений.

4. Практика по шагам

Шаг 1. Запустить цель и проверить доступность

Зачем: DAST всегда начинается с понятной и разрешённой цели.

Что сделать:

1. Запустите python -m backend.app в vulnerable-app.

2. Во втором терминале проверьте /health.

3. Убедитесь, что используется локальный адрес и учебная БД.

4. Создайте reports.

Команда или код:

```
python -m backend.app
# Во втором терминале:
mkdir -p reports
curl http://127.0.0.1:5000/health
```

Ожидаемый результат: health возвращает status ok. Приложение доступно на 127.0.0.1:5000, а из Docker — host.docker.internal:5000.

Шаг 2. Выполнить ZAP Baseline

Зачем: Получим пассивные наблюдения без активных атак.

Что сделать:

1. Запустите Docker с zap-baseline.py.

2. Дождитесь окончания spider и passive scan.

3. Откройте reports/zap-report.html.

4. Выпишите CSP, X-Content-Type-Options, anti-CSRF, XSS и другие WARN.

Команда или код:

```
docker run --rm -v "${PWD}/reports:/zap/wrk/:rw" -t ghcr.io/zaproxy/zaproxy:stable zap-
  baseline.py -t http://host.docker.internal:5000 -r zap-report.html -I
```

Ожидаемый результат: Появляется HTML-отчёт. В контрольном запуске было 10 групп WARN, включая потенциальный XSS в /search. FAIL-NEW: 0 не используем как доказательство безопасности.

Если не получилось:

• Connection refused: приложение не запущено или неверный порт.

• Нет отчёта: проверьте, что смонтирован каталог reports и дождитесь завершения.

Шаг 3. Выполнить active scan и оценить покрытие

Зачем: Активные проверки нужны для SQLi и XSS. Это может занять больше десяти минут.

Что сделать:

1. Запустите zap-full-scan.py только на локальной цели.

2. Дождитесь формирования zap-full-report.html; не прерывайте процесс сразу после spider.

3. Найдите SQL Injection, Cross Site Scripting (Reflected) и (Persistent).

4. Сравните список посещённых URL с маршрутами приложения.

Команда или код:

```
docker run --rm -v "${PWD}/reports:/zap/wrk/:rw" -t ghcr.io/zaproxy/zaproxy:stable zap-full-
  scan.py -m 1 -T 10 -t http://host.docker.internal:5000/login -r zap-full-report.html -I
```

Пример и объяснение: Для полного сайта замените URL на http://host.docker.internal:5000. Это дольше. Любые активные проверки выполняются только на собственной учебной цели.

Ожидаемый результат: Отчёт содержит SQL Injection и XSS High. Если маршрут не посещался, отсутствие находки по нему ничего не доказывает.

Шаг 4. Подтвердить три типа дефектов вручную

Зачем: Ручная проверка показывает, что вы понимаете причину и влияние.

Что сделать:

1. Для SQLi повторите вход admin' -- с любым паролем на уязвимой копии и объясните результат.

2. Для XSS введите training и затем локальный alert-пример, если преподаватель разрешил. Не используйте внешние сайты.

3. Для IDOR выйдите из сессии и откройте /profile?id=1.

4. Запишите запрос, ответ, наблюдаемое влияние и ограничения доказательства.

Пример и объяснение: IDOR доказывается не словом «admin», а отсутствием проверки: неавторизованный запрос получает email и secret_note. Для чужого id нужен тест 403 после исправления.

Ожидаемый результат: До исправлений обход входа возвращает redirect, HTML не экранируется, профиль admin доступен без сессии. После ЛР2 SQLi/XSS уже должны отличаться — сравните.

Шаг 5. Исправить контроль доступа и заголовки

Зачем: Сканер не исправит бизнес-правило. Добавим проверку владельца на backend.

Что сделать:

1. В login после session['username'] добавьте session['role'] = user[2].

2. В profile проверьте сессию и сравните requested id с session user_id.

3. Добавьте after_request с заголовками.

4. Добавьте тесты: без сессии 401, чужой id 403, свой профиль 200.

5. Повторите pytest и ZAP Baseline.

Команда или код:

```
from flask import abort

# В profile перед запросом к БД:
if 'user_id' not in session:
    abort(401)
requested_id = int(user_id)
if session['user_id'] != requested_id and session.get('role') != 'admin':
    abort(403)

# После создания app:
```

```
@app.after_request
def security_headers(response):
    response.headers['Content-Security-Policy'] = "default-src 'self'; frame-ancestors 'none';
      base-uri 'self'; form-action 'self'"
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['X-Frame-Options'] = 'DENY'
    return response
```

Пример и объяснение: В login добавьте: session['role'] = user[2]. CSP в реальном проекте строят под конкретные ресурсы; anti-CSRF требует отдельной защиты форм.

Ожидаемый результат: Без входа профиль возвращает 401; студент не видит чужой id; admin действует по явной политике. Заголовки появляются в ответе.

Шаг 6. Собрать единый CI/CD и принять решение о релизе

Зачем: Финальный результат — не набор сканеров, а понятный процесс с владельцами и повторными проверками.

Что сделать:

1. Нарисуйте PR → Gitleaks + Semgrep + Trivy → tests → staging → ZAP → SBOM/release.

2. Для каждого этапа укажите команду, отчёт, gate и владельца.

3. Составьте реестр: исправлено, осталось, принято, блокирует релиз.

4. Опишите, что происходит при ошибке сканера и при временном исключении.

Пример и объяснение: Gate: подтверждённый секрет — блокировать; новая SQL injection — блокировать; WARN о заголовке — разобрать и исправить по политике; ошибка ZAP — проверка не выполнена.

Ожидаемый результат: Есть схема pipeline и обоснованное решение о выпуске. Оставшиеся риски: пароли в открытом виде, CSRF и другие — не скрыты, а внесены в план.

5. Что сдать в отчёте

• Команды запуска цели и ZAP.

• Таблица Baseline/active и покрытия.

• Ручные доказательства SQLi/XSS/IDOR.

• Код контроля доступа, заголовки и тесты.

• Схема CI/CD, gates и решение о релизе.

В конце добавьте краткий вывод: какой дефект был найден, какая причина устранена, что проверено повторно и какие риски остались.

6. Проверь себя

1. Чем Baseline отличается от active scan?

2. Почему FAIL: 0 не доказывает безопасность?

3. Почему IDOR требует ручной проверки?

4. Что подтверждает повторный запуск после исправления?

5. Почему зелёный pipeline не равен соответствию ГОСТу?

7. Критерии оценивания

Критерий	Баллы

Baseline и покрытие	20
Active scan и ручной разбор	20
Контроль доступа и заголовки	25
Повторные проверки	15
CI/CD и отчёт	20

Максимум — 100 баллов. За использование чужой системы без разрешения или публикацию настоящих секретов работа не принимается. Честно описанный отрицательный результат и ограничения инструмента являются нормальной частью анализа.

8. Источники

• ГОСТ Р 56939-2024: https://www.gostinfo.ru/catalog/Details/?id=8094263

• ГОСТ Р 71207-2024: https://protect.gost.ru/gost/details/9dd6f064-6014-4416-a3ca-ef72ce0a6c3a

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

• ZAP Baseline: https://www.zaproxy.org/docs/docker/baseline-scan/